CSE 511 Data Processing at Scale - Spring 2025
Project-1 Final Report

Neha Valeti
Arizona State University

1 Introduction

There's a lot of data generated by modern cities everyday from taxi rides to traffic flows to socila networks and sensor readings. All this data is form relationships and to make sense of such relations is important to build smarter transportation systems, optimizing infrastructure of the cities and solving problems in urban environments through discovering patterns in such huge amoutns of data. However, the traditional databases are not efficient to handle such highly-connected data, as these databases rely on rigid table structures, which leads to performace bottlenecks and complex query logic.

Graph-based solutions addresses this challenge by representing data as nodes(entities) and edges(relationships), allowing for faster traversals, simpler queries and more efficienct database systems. In this project the main goal is to use graph based mdelling for real worl internactions and to use New York City taxi trips as a network of pickup and dropoff locations (Location nodes) connected by trip events (TRIP relationships). This structure helps in capturing the flow of thraffic across different parts of the city, enabling advanced graph algorithms such as Page Rank - to find the most influential hubs and Breadth-First Search to discover paths between locations.

This project i set out to tackle the challenges of the scalable processing by designing, building a real-worl system using Neo4j for graph storage and analytics, Kafka for real-time data streaming, and Kubernetes or orchestration and scalability The project is divided into two phases:

- Phase 1: which is focused on developing self-containes Dockerized Neo4j Environment, loading real-world taxi trip data from the NYC Yellow Cab dataset, and implementing core graph algorithms like PageRank and BFS to extract meaningful insights from static data.

- Phase 2: Which extends the system into a fully distributed, cloud-native pipeline by integrating Kafka, Kubernetes, and Kafka Connect — enabling real-time ingestion, dynamic graph construction, and continuous analysis as new trip data streamed in.

2 Methodology

This section outlines the systematic approach followed b=to design, implement, and deploy a scalable graph-based data processing pipeline. The methodology is divided into two major phases: 1) local graph database setup and algorithm implementation and 2) distributed real-time data ingestion and graph construction using a cloud-native architecture

2.1 Phase 1: Local Neo4j Deployment and Graph Algorithms

2.1.1 Data Preprocessing

The NYC Yellow Taxi dataset for March 2022 was selected as the primary data source. Using a Python script (`data_loader.py`), the dataset was filtered to include only trips with Bronx locations. The pick-up and drop-off timestamps, the distance traveled and the amount of the fare were extracted.

2.1.2 Docker Environment for Neo4j

A Dockerfile was created to automate the deployment of a self contained Neo4j database instance. Then a neo4j container is created and it included the following steps:

- Firstly, install all the required Python dependencies and packages like Neo4j, Pandas, pyarrow for data manipulation and communication with the database. Additionally, Neo4j was installed from the offical Debian Repository.

- Authentication was configured for Neo4j, setting a predefined password, modifying the remote access settings to enable external connnections.

- The Neo4j GDS plugin was downloaded directly from the official release repository and placed into the Neo4j's plugin directory. Proper permissions was given for running the container.

- A separate import directory was created at /var/lib/neo4j/import to load the data while running the container

- The project files from github repository account was cloned into the container folder and the parquet file for the data is also downloaded. With the help of this docker file (`data_loader.py`) file the data was loaded into Neo4j as a graph where nodes represent unique pickup and dropoff locations and relationships (TRIP type) encoded trip distance, fare, pickup time and dropoff time. The Phase 1 system architecture is illustrated in Figure 1, showing the local machine, Docker Container, Neo4j instance, and the flow of data loading and querying.

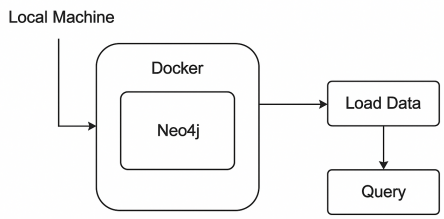

Figure 1: Phase 1 Architecture.

2.1.3 PageRank and Breadth-First Search (BFS) algorithms

Two fundamental graph algorithms were implemented using the GDS library

- PageRank: Using PageRank, the goal was to measure how important each Location node was based on the number and strength of trips connected to it, similar to identifying major traffic hubs within the Bronx.
- Breadth-First Search (BFS): the idea was to find the shortest paths between a starting location and several target locations, helping visualize how different parts of the network were connected.

Both algorithms were run externally using a Python script (`interface.py`), which connected to the Neo4j database through the Bolt protocol over exposed ports.

2.2 Phase 2: Using Kubernetes, kafka and Zookeeper

2.2.1 Kubenetes Cluster setup

Firstly, all the required plugins and dependencies were installed, Minikube and Kubectl were installed. Then a Kubernetes environment was provisioned locally using Minikube. To ensure that everything worked correctly scripts from the `grader.md` which helped configure the environment and ensured all requires ervices were ready.

2.2.2 Kafka and Zookeeper Deployment

The first step is to install the kafka and Zookeeper in the kubernetes cluster using the YAML files (`kafka-setup.yaml` and `zookeeper-setup.yaml`)

- Zookeper is responsible for keeping track of the kafka browsers and making sure that they are coordinated well.
- Kafka Broker then handles sending and receiving messages or data between different parts of the system.
- Kafka is mainly setup to accept the plain connections and create new topics automatically when needed.

This step is done using the `zookeeper-setup.yaml` and `kafka-setup.yaml` file

2.2.3 Neo4j Deployment using Helm

After the zookeper and kakfka setup was completed , the Neo4j graph database using helm is deployed, helm is a tool which simplifies installing application on kubernetes.This is done using `neo4j-values.yaml` and `neo4j-service.yaml` files. In the neo4j values file code is written to agree the neo4j license items, set passwords for database access, enable the graph datasscience plugin to run graph algorithms.CPU and memory limits were assigned to manage resources.Neo4j service file is used to open Neo4j's important ports inside the kubernetes cluster.

2.2.4 Kafka and Neo4j Sink Connnections

Next step is to setup up kafka connection which acts as bridge between Kafka and Neo4j. For which a `kafka-neo4j-connector.yaml` file was created to reploy the connect pod that automaticall reads messages from the kafka topic which is nyc-taxicab-data. And inserts them into the Neo4j database by using a predefined Cypher query. The connector settings were stored in the `sink.neo4j.json` file, and a startup script `init.sh` was used to register the connector as soon as Kafka Connect became ready.

2.2.5 Data Producer Simulation

Finally, a Python script data-producer file is written to simulate real-time data streaming.The script contains the following operations: Reads Bronx trip data from a Parquet file. Converts each trip into a JSON message. Sends each message to the Kafka topic nyc-taxi-data. Figure 2 shows the Architecture diagram for the kafka, Neo4j, Kubernetes connection. This created a continuous stream of live trip records, which Kafka Connect then inserted into the Neo4j database automatically.

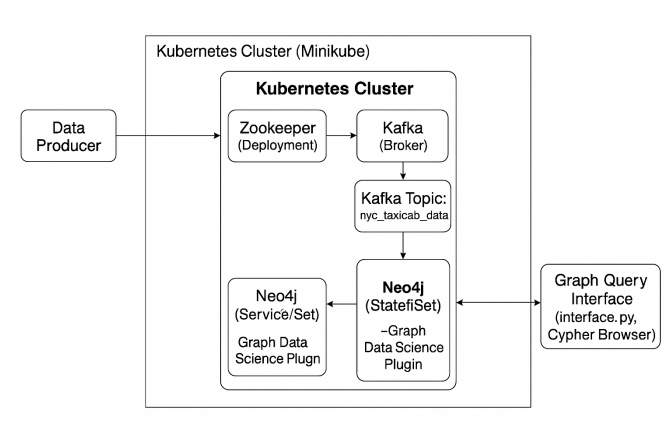

Figure 2: Phase 2 Architecture.

3 Results

The project sucessfully demonstrated the development of a scalable, real-time graph processing system. Key Findings Include

3.1 Phase 1

- Data Loading and Graph Creation: In phase 1, a Docker Container was built in the local environment where Neo4j was set up. The entire dataset was loaded into Neo4j. And each record from the dataset created two types of graph elements which are location nodes for pickup and dropoff points. These also include Trip relationships connecting two locations, including trip distance,fare, and timestamps.
- Graph Algorithms (Page Rank and BFS): After loading the data, two main algorithms were run
  - Page Rank helped helped identify the most important locations (hubs) based on how connected they were to other locations.
  - Breadth-First Search (BFS) calculated the shortest paths between a given starting point and multiple destinations, showing how locations were linked across the city.

Figure 3 shows the cypher query and the graph for the phase 1 execution.

3.2 Phase 2

In Phase 2, the project transitioned from static data loading to real-time streaming. Taxi trip records were continuously sent to a Kafka topic (nyc-taxicab-data), and Kafka Connect automatically passed this data into Neo4j, updating the graph

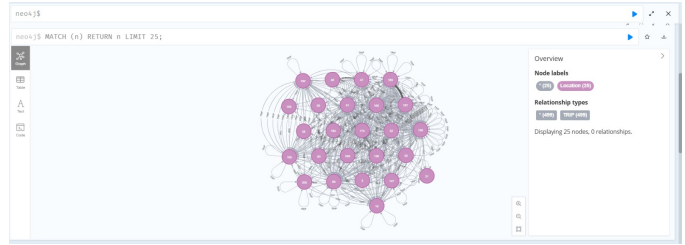

Figure 3: Graph in Neo4j.

as new trips arrived. The entire pipeline—from the Kafka producer to Neo4j—ran smoothly without interruptions, demonstrating the system's ability to handle live data reliably. All components, including Zookeeper, Kafka, Kafka Connect, and Neo4j, were successfully deployed in a Kubernetes cluster using Minikube. ClusterIP services and port forwarding were used to ensure proper communication, and the setup proved stable and well-orchestrated. Figure 4 shows the output for

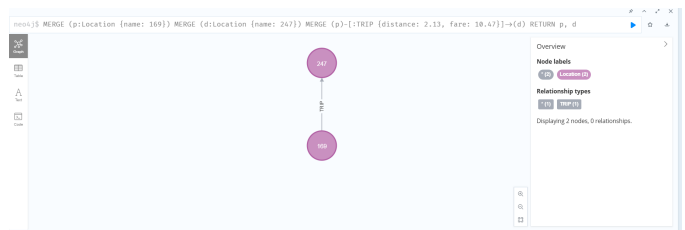

Figure 4: Node connection Graph in Neo4j.

the Cypher query and the graph for phase 2

4 Discussion

This project highlighted the advantages and difficulties of creating a distributed, real-time graph analytics system. A major benefit was the modular configuration with Docker and Kubernetes, allowing for easy isolation of components, service scaling, and environment resets when necessary. Kafka was instrumental in ensuring the data stream was handled efficiently, with little to no delay or loss of messages. Neo4j's Graph Data Science plugin made it easier to execute intricate graph algorithms on extensive datasets. Conversely, there were several obstacles throughout the journey. Establishing effective communication among Kubernetes pods necessitated meticulous configuration of services and networks. Kafka Connect required adjustment to manage the timing between the initiation of the producer and the readiness of Neo4j to accept data. The initial configuration of Neo4j Docker required additional effort, particularly regarding authentication and the activation of plugins. In the future, this system could see enhancements through various methods: integrating sophisticated graph algorithms (such as community detection

or shortest weighted paths), expanding Kafka with additional brokers, and incorporating monitoring tools like Prometheus or Grafana to oversee system performance. Improvements like data validation and immediate filtering prior to ingestion may also enhance the quality and utility of the data. In summary, this initiative establishes a solid base for developing more sophisticated, production-ready solutions for smart city analysis.

5 Conclusion

This project effectively utilized Docker, Kafka, Neo4j, and Kubernetes to create a scalable, real-time system for managing graph-oriented data. It started with a simple configuration in Phase 1, where Neo4j was containerized and utilized for loading static data. During Phase 2, the system evolved into a completely distributed pipeline operating on Kubernetes, able to manage live data streams and refresh the graph in real time.

Through meticulous design and integration of each element, the project demonstrated how actual transportation data, such as NYC taxi rides, can be efficiently modeled, streamed, and examined with graph technologies. The structure and experience established here provide a strong foundation for upcoming endeavors in fields such as smart cities, logistics, and real-time recommendation systems.

References

1. A. Lopes, D. Rodrigues, J. Saraiva, M. Abbasi, P. Martins, and C. Wanzeller, "Scalability and performance evaluation of graph database systems: A comparative study of Neo4j, JanusGraph, Memgraph, NebulaGraph, and TigerGraph," in Proc. 2023 2nd Int. Conf. on Smart Technologies for Smart Nation (SmartTechCon), Aug. 2023, doi: 10.1109/SmartTechCon57526.2023.10391694.

2. K. Padmanaban, T. R. Ganesh Babu, K. Karthika, B. Pattanaik, D. K., and C. Srinivasan, "Apache Kafka on big data event streaming for enhanced data flows," in Proc. 2024 8th Int. Conf. on I-SMAC (IoT in Social, Mobile, Analytics and Cloud), Oct. 2024.

3. R. Hiraman Bhole, M. Viresh Chapte, and C. Abhijeet Karve, "A study of Apache Kafka in big data stream processing," in Proc. 2018 Int. Conf. on Information, Communication, Engineering and Technology (ICICET), Pune, India, Aug. 2018, doi: 10.1109/ICICET.2018.8533771.

4. Y. Pan, I. Chen, F. Brasileiro, G. Jayaputera, and R. Sinnott, "A performance comparison of cloud-based container orchestration tools," in Proc. 2019 IEEE International Conference on Big Knowledge (ICBK), Beijing, China, 2019, pp. 191–198, doi: 10.1109/ICBK.2019.00033.

5. G. Xie et al., "AttractRank: District attraction ranking analysis based on taxi big data," IEEE Trans. Ind. Informatics, vol. 17, no. 10, pp. 6833–6842, Oct. 2021, doi: 10.1109/TII.2020.2994038.

6. V. Beneš and M. SvíteK, "Knowledge graphs for smart cities," in Proc. 2022 Smart City Symposium Prague (SCSP), Prague, Czech Republic, May 2022, pp. 1–6, doi: 10.1109/SCSP54748.2022.9792541.

7. Z. Zheng and A. K. Bashir, "Graph-enabled intelligent vehicular network data processing," IEEE Trans. Intell. Transp. Syst. (early access), 2022, doi: 10.1109/TITS.2022.3158045.